Page No :
Date :

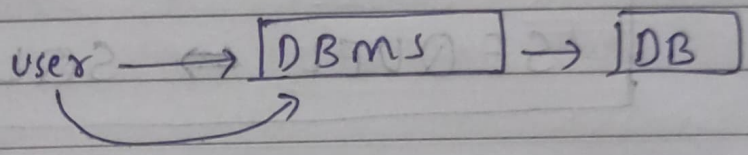
SQL

Database:- "Collection of data" in a format, that can be easily accessed (updation, deletion, adding etc....)

→ format is "digital"

Database Mangement System (DBMS):-

A Software application used to "manage our DB" is called DBMS.



We use "SQL" for interacting with DBMS, which manages our database.

✱ Types of Databases:-

① **Relational:-** Data stored in tables.

Eg:- MySQL, ORACLE, SQL Server

② **Non-Relational (NO SQL):-** Data not stored in table Eg:- mongoDB

→ We use [SQL] to work with relational DBMS

Page No:
Date:

⊛ SQL (Structured Query language):-

SQL is a programming language used to interact with relational databases.

It is used to perform CRUD operations.

Create, Read, Update, Delete

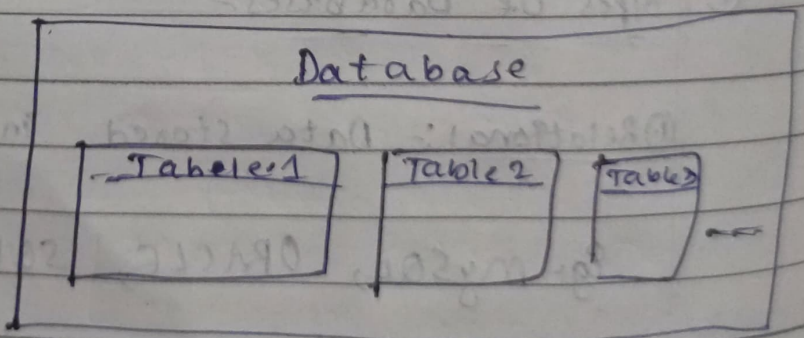
SEQUEL ⇔ SQL

Both are same

SEQUEL
- Structured
- English
- QUEry
- Language

SQL
- Structured
- Query
- Language

Database Structure:



Eg:- School Database

- Students
- fees
- faculty
- Staff
- etc...

Page No :
Date :

Table :-

Roll No	Name	Class	Marks
1	-	-	-
2	-	-	-
3	-	-	-
4	-	-	-
5	-	-	-

→ row 1
→ row 2
→ row 3

col 1 col 2 col 3 .....

Columns → Structure / schema (design)
Rows → Individual data

Common ds :-

1. Create data base :-

```
CREATE DATABASE db-name;
```

Eg :- CREATE DATABASE school;

must for command.

school database is created.

Note :- SQL is "non-case sensitive language".

→ We can write it in ~~small~~ lower (or) upper case lang.

```
Create database db-name;
```
(✓) valid.

Create Database db-name; (✓) valid.

upper, lower case.

Page No:
Date:

*** USE db_name;

Eg:- school, college, bank are our databases.

USE school; → when we execute this command, the further operations we perform will happen for school database.

① CREATE TABLE:-

Syntax:-

```
CREATE TABLE table_name (
    column1  datatype  constraint,
    column2  datatype  constraint,
    column3  datatype  constraint
    :        :         :
);
```

column name

④ Data types:- [(mandatory) must be provided, otherwise it columns doesn't work]

(a.) Numeric Datatype:

1. INT → integer values.
2. SMALL INT → small integers.
3. BIG INT → very large integers.
4. FLOAT → floating numbers.
5. DECIMAL (P,S) → DECIMAL (10, 2)
   max. it holds 10 number

Page No :
Date :

DECIMAL (10, 2) :-

maximum holds (10) numbers, in which (2) numbers need be in decimal place.

Eg: 1.23, 10.00, 273.33, 12345678.90
all are valid.

(b.) Character / String Data types

(1.) CHAR (n) → fixed length string
Eg:- CHAR (10) = always 10 characters.

(2.) VARCHAR (n) → Variable length string (up to n characters).

(3.) TEXT → large Variable length text.
simply, VARCHAR (max).

(c.) Date and Time

(1.) DATE → stores only date (YYYY-MM-DD)
(2.) TIME → stores only time.
(3.) DATETIME → stores date and time.
(4.) SMALL DATETIME → " " " "
with lower range.

(2.) Other Datatypes:- BIT (1) ⇒ 0, 1
BIT (2) ⇒ 00, 01, 10, 11.
BIT (3) ⇒ 001, 010, 100 ...

① BIT → Boolean (0 (0) 1).
② BINARY → fixed length binary data.
③ VARBINARY → variable " " "
④ IMAGE → Binary large object
BLOB (100) → (similar to VARBINARY(MAX))

* Constraints:- [optional when creating tables, with usage accuracy will come] to database

Specific rules for data in table. →

① PRIMARY KEY:- ensures unique and non-null values.
Eg:- StudentId INT PRIMARY KEY

② FOREIGN KEY:- Links one table to another table.

Eg:- FOREIGN KEY (DEPT Id) REFERENCES DEPARTMENT (DEPT Id)

→ ③ UNIQUE:- Ensures all values in columns are different.

Eg:- Email VARCHAR (100) UNIQUE

→ ④ NOT NULL:- prevents empty values.

Eg:- First Name VARCHAR (50) NOT NULL

Page No:
Date:

5. CHECK: Ensures a condition is satisfied.

eg: Age INT CHECK (Age >= 18)

condition.

6. DEFAULT:- Assigns default value, if no value is provided.

eg:- City VARCHAR(50) DEFAULT 'Delhi'

If no city is mentioned; 'Delhi' will be stored.

- CREATE TABLE:-

USE School; (using school database)

CREATE TABLE students (

Id INT PRIMARY KEY,

name VARCHAR(50) NOT NULL,

age INT

);

column names

data types.

constraints.

no constraint (as it is optional).

O/P: Students

Id	name	age

Page No:
Date:

(*) Adding data into our table; we use

INSERT INTO

INSERT INTO students VALUES(1, "Ramesh", 2...
INSERT INTO students VALUES(2, "Vinat", 3...
: | { (;
Table Name. values to be added

To get our table:-

SELECT * FROM Student;

all

Diff b/w CHAR (&) VARCHAR:

CHAR (50)	VARCHAR (50)
Entry: Ramesh	Entry: Ramesh.
In, "CHAR" the, no. of memory space is fixed.	In "VARCHAR", only the required amount of space will be taken.
R A M E S H ... D D (50 times)	R A M E S
1 2 3 4 5 6 ---- upto 50	only "6" characters is used here. No padding to remaining text.
Here, only, 6 Char space is used, the remaining 44 will be padded with blancks, so; it takes fixed memory space, even if it used(0) not.	

Page No :
Date :

Few more data types:-

signed :- allow both negitive and positive values. (-128 to 128)

unsigned :- allows only zero and positive values, but increase the maximum upper range.

[0 to 258]

TYPES of SQL commands:-

1. DDL (Data Definition Language):-
   CREATE, ALTER, RENAME, DROP, TRUNCATE

2. DQL (Data Query Language):- SELECT

3. DML (Data Manipulation lang):-
   INSERT, UPDATE (&) DELETE

4. DCL (Data Control language):- grant (&) revoke,
   permission to users

5. TCL (Transaction Control Language):-
   Rollback, start transaction, Commit etc..

Page No:
Date:

Queries for best practices:-

(1.) IF NOT EXISTS:-

If we have a Database name "SCHOOL". Again, if we want to create the same DB, unintentionally. It throws an Error.

Eg:- CREATE DATABASE SCHOOL;

O/P:- [ERROR:] Database already exists.

By using IF NOT EXISTS:

CREATE DATABASE IF NOT EXISTS School;

O/P: [Warning : DB already EXists.]

To handle Errors, it is good to use IF NOT EXISTS.

(2.) IF EXISTS:-

while deleting, to overcome Errors, we use it.

Eg: DROP DATABASE db-name;

O/P: (Error) If db not Exists

→ DROP DATABASE IF EXISTS db-name;

O/P:- (Warning) → If db not Exists.

Page No:
Date:

(3) SHOW DATABASES → Show all the databases

(4) SHOW TABLES → Show all the tables of database, which we are using

Eg:- USE SCHOOL;
SHOW TABLES;

O/P:- Students, Staff } → Tables in SCHOOL DB.

TABLE Related Queries:-

(1) create table:

```
CREATE TABLE table_name (
    col_name_1 datatype constraint,
    col_name_2 ...  ...  ...  ,
    ...  ...  ...  ...
);
```

(2) TO READ:- (SELECT)

```
SELECT * from table_name;
       ↓
      all
```

(3) INSERTING VALUES:-

~~INSERT INTO table_name~~ → Student

```
INSERT INTO table_name
  (column_1, column_2)
VALUES (col_val1, col_2, val_2),
       (col_val1, col_2, val_2);
```

Page No :
Date :

Eg:- INSERT INTO Student
(rollno, name)
VALUES (101, "VENU");
(102, "VAMSI"),
(103, "Ram");

Here, mentioning columns are optional;
If we don't want to change order of inserting values, we don't have to mention columns.

INSERT INTO Students Values (101, "venu");
↓
order of rollno, name

But, if we change order, like ("venu", 101);
↓
It throws an Error, as "venu" can't be an INT.

If we use; (name, rollno) VALUES ("venu", 01),
Now, the values are inserted.

TYPES OF KEYS:-

1. PRIMARY KEY:- It is used to uniquely identify each record in a table.
→ no Null values are allowed.

Primary key is not mandatory in table; but if it is used for a column, it has to be unique & not null.

Page No:
Date:

Key points:
1. Every table has only one Primary key.
2. It may consists of single column (or) combination of columns.

single column as primary key:

```
id INT PRIMARY key
)
```

Here, the id's in table, has to be unique.

multiple columns as primary key:-

```
CREATE TABLE temp1 (
    id INT,
    name VARCHAR(50),
    age INT,
    PRIMARY KEY (id, name)
);
```

Primary key will assigned at last.

Here, (id, name) as combination we took it as primary key.

Means;

id	name
1	Ram
1	Kiran
2	Kiran
2	Manju

Similar id's but not an error.

Here, id's (&) names have been repeated but combination should'nt repeated.

Eg: (1 Kiran) → Error; as combination should be unique.

Page No :
Date :

(2.) FOREIGN KEY: It is used to link two tables together.

→ It establishes a "referential relationships" blw a column in one table and the primary key of another table

(*) Properties of Foreign Key:

(1.) Can accept duplicate values.
(2.) " " NULL ".
(3) Ensure that inserted values must already exists in the parent table's primary key

Eg:

~~Table 1~~

Place_Info

Primary key id	Place
1	Pune
2	Punjab
3	Delhi

(unique & not null)

Stu_Info.

Primary key Stu_id	name	Place
1	"Ravi"	1
2	"Rani"	2
3	"Ramesh"	1
4	"Somu"	3
5	"Sunny"	2

Here, in Place table.
The primary key ~~of table 1~~ are unique & not null,
Now, it is used in Student_Info, id have repeted; ~~but~~ & null, still it will accept.

Page No:
Date:

in Stud-Info

Here, "place" column acts as "Foreign key" as it references to primary key of Place table.

eg:- CREATE TABLE PLACE-Info (
id INT PRIMARY-KEY,
Place VARCHAR(50)
);

CREATE TABLE stu-Info (
Stu-id INT PRIMARY-KEY,
name VARCHAR(50),
Place VARCHAR(50),
FOREIGN KEY(Place) REFFERENCES PLACE-Info (Place)
);

Here, Stu-id = PRIMARY KEY
Place = Foreign KEY

④ "CHECK" Constraint:- Input values had to follow some parameter.

① Single check:-

CREATE TABLE temp-1 (
age INT CHECK (age >= 18)
);

→ all input user age should be greater than 18.

Page No:
Date:

④ multiple constraint (checks):-

```
CREATE TABLE City (
    Id INT PRIMARY KEY,
    City VARCHAR (50),
    age INT,
    CONSTRAINT age-check CHECK
        (age >= 18 AND City = "Delhi")
);
```

→ user details has to be age >= 18 (&) City = Delhi, otherwise it throw an error.

⑤ SELECT :-

Basic Syntax:

```
SELECT Col-1, Col-2 FROM Students;
       Id     Place
```

Id	Place

olp:-

We just use, we want, (1 to all) (demy

→ SELECT * FROM students;

all columns (so whole table comes).

Page No:
Date:

1. ⊛ DISTINCT:- To filter and get unique values from the particular columns.

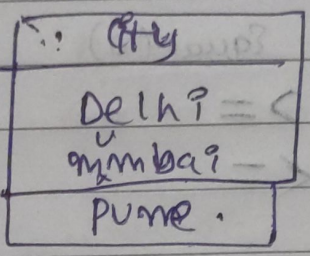
Eg:- SELECT DISTINCT city, FROM Students;

↓

It provide city column, but only unique values.

Eg:- If there are 2 Delhi, 2 mumbai, 2 Pune.

O/P:-



⊛ "WHERE" CLAUSE:- a particular condition should be satisfied.

Eg:- SELECT * FROM Students WHERE age > 20;

O/P:- all students whose age > 20.

Multiple clauses:

Eg:- SELECT * FROM student WHERE (age >= 18 AND place = "Delhi");

O/P:- Ues which satisfy both of these condition.

Page No:
Date:

✱ Operators in WHERE clause:-

① Arithmetic Operators:- + (Add), − (Sub), * (multi), / (Division), %o (Modules)

② Comparission operators:- (i) = (equal to)
eg: WHERE Marks = 90;

(ii) != (not equal to)
(iii) > , >=
(iv) < , <=

③ Bitwise operators:- & (Bitwise AND), | (Bitwise OR).

④ Logical Operators:-

① AND:- WHERE cond-1 [AND] cond-2
Both conditions should satisfy.

② OR:- WHERE cond-1 [OR] cond-2
any one condition satisfies,

③ BETWEEN:- (Range) eg value [80, 81 --- 90] is satisfied Inclusive

WHERE marks BETWEEN 80 and 90;

④ IN:-

SELEC

If any

eg:

⑤ NO

O/P:-

Page No :
Date :

④ IN :- (matches any value in the list).

SELECT * FROM Student WHERE City IN ("Delhi", "mumbai", "Pune");

↓ (this) list, if any user's city is in would be printed.

eg:

Id	Place
1	Pune
2	Banglore
3	Delhi
4	Chennai

o/p:

1	Pune
3	Delhi

↓

These values are from given list for City

⑤ NOT :- (to negate the given condition)

User's details shouldn't satisfy this condition

WHERE City NOT IN ("Delhi", "mumbai", "Pune");

o/p:- same eg:

Id	Place.
2	Banglalore
4	Chennai

few more → ALL, LIKE, ANY

Page No :
Date :

(*) LIMIT Clause :-

Set an "upper limit" on number of rows to be returned.

```
SELECT * FROM Student LIMIT 3;
```

(01)
col-1, col-2 ...

o/p :-

id
1
2
3

first 3

~~(**)~~ WHERE CLAUSE + LIMIT CLAUSE :-

```
SELECT * FROM student
WHERE marks > 80
LIMIT 3;
```

o/p :-

id	marks
2	83
3	96
6	90

Here, marks are greater than 80 (2) only & return (3) user details from all among them.

Page No :
Date :

✱ Order by Clause:-

To sort in ascending (ASC) or descending order (DESC)

ASC:-

```
SELECT * FROM student ORDER BY
                    City ASC;
```

City in table, Ascending [A - B - C - D -]

Eg: O/P:-

Id	place
3	Delhi
1	Mumbai
2	Pune

D -- M -- P (ascending order)

DESC:- TO "3" highest mark students.

```
SELECT * FROM student        (highest to lowest)
    ORDER BY marks DESC
    LIMIT 3;
```

O/P:

Id	marks
5	96
3	93
1	90

Page No:
Date:

(*) Aggregate Functions:-

Aggregate functions performs a calculation on a set of values, & return a single value.

1. count () → finding the count of given value
2. MAX () → maximum among of " "
3. MIN () ⇒ minimum " " " "
4. SUM () → SUM of all numbers.
5. AVG () ⇒ Average of all numbers.

eg:- ~~MAX~~

```
96      SELECT MAX(marks) FROM Student;
16      "      MIN(marks)  "  "  ;
746     "      SUM(marks)  "  "  ;
73.33   "      AVG(marks)  "  "  ;
6       "      Count(~~***~~ id)  "  "  ;
```

↓

as id's are distinct (&) not null; it give Proper count of users.

***** GROUP BY:- Group By clause is used to "arrange identical data into groups!"

→

→ It is combined with aggregate functions like COUNT(), SUM(), AVG(), MAX() etc...

eg:-

Page No:
Date:

eg:- (1) SELECT city, count(name) (or) count(id)
FROM student
GROUP BY city;

↓

all the ~~indivi~~ unique cities will be taken

(2) find the count of student who are from that cities.

o/p:-

~~city~~ City ~~number~~	count(id)	
Pune	3	→ 3 students from Pune
mumbai	2	→ 2 " " Mumbai
Delhi	1	→ 1 " " Delhi

(2) ~~If the table~~ ~~select~~ ~~multiple~~

SELECT city, name, COUNT(roll no)
FROM student
GROUP BY city;

o/p:- ERROR - (1055)

Note:- When "GROUP BY" is used, every column in the SELECT statement must either:

(1.) Be included in the "GROUP BY" clause

(or)

(2.) Be used inside an aggregate function (Sum, Count, Avg Etc...

→ If we select extral columns, not grouped (or) aggregated, it throws an error.

Page No:
Date:

Eg:- SELECT city, name, count(rollno)
FROM student
GROUP BY (city, name);

O/P:-

city	name	count(rollno)
Pune	anil	1
Mumbai	Kumar	1
Mumbai	Kranthi	1
Delhi	vishnu	1
Delhi	vaibhav	1
Pune	varun	2

This group has only 1 in entire table.

There are 2 times persson named "Varun" from "Pune".

SELECT city, Avg(marks)
From Student
group By (city);

Average of marks for each city students.

Practice:- write Query to find avg marks in each city (ascending order)!

(A.) SELECT City, Avg(marks)
FROM student
GROUP BY City
ORDER BY City;

Based on City

O/P:-

City.	Avg(marks)
Delhi	93.80
mumbai	83.90
Pune	96.30

Order BY Based on Avg(Marks)

ORDER By AVG(marks);

O/P:-

City	Avg.
Pune	83.40
Delhi	93.80
Pune	96.30

Page No :
Date :

(2.) From give table, find the total payment (count) acc to each payment method.

Payment

customer_id	customer	mode	city
101	—	NetBanking	—
102	—	Credit	—
103	—	NetBanking	—
104	—	Debit	—
}			

Opt → group → NetBanking, Credit, Debit

```
SELECT mode, COUNT (customer)
FROM payment
GROUP BY mode;
```

(*) Having Clause :-

Similar to WHERE i.e., applies some "condition on rows".

→ used when we wanted apply any "condition after grouping"

Eg: Count no. of students city where max (cross 90 marks)

```
SELECT  count (name), city
FROM    student
GROUP BY city
HAVING  max (marks) > 90;
```

Page No :
Date :

O/P from Grouped by table; It will take the maximum marks of student should by more than 90.

Before Having:

City	Max (Marks)
Delhi	85
Chennai	76
Kolkata	88
Mumbai	92
Bangalore	95

After Having max(marks) > 90;

City	max (marks)
Mumbai	92
Bengaluru	95

GENERAL ORDER :-

```
SELECT   column(s)
FROM     table_name
WHERE    condition
GROUP BY column(s)
HAVING   condition
ORDER BY column(s) ASC;
```

SELECT … GROUP BY ⇒ This will give a table.

HAVING [Perform this Having condition(s) after GROUP BY]

ORDER BY … ASC; ↓ finally ordering.

Page No:
Date:

Update:- For updating existing rows:

```
UPDATE table-name
SET col-1 = val1, col-2 = val2
WHERE condition;
```

~~updating~~

SAFE UPDATE:- safe update mode is a feature that "prevents accidental updates (&) deletion of rows in table."

If SAFE UPDATE mode is enabled, running UPDATE (&) DELETE without proper condition will throw an error,

"TO Disable" SAFE UPDATE:-

```
SET SQL - SAFE-UPDATES = 0;
```

Now, user can update's (&) deletes by their interest, once every thing is done, we can again enable it.

TO ENABLE:-

```
SET SQL - SAFE-UPDATES = 1;
```

Page No:
Date:

Eg ①: I want ~~change the~~ g to add "2" more marks to students who got 90+ marks.

SOL:

```
SET SQL_SAFE_UPDATE = 0 ;
```

To get access → UPDATE Students
Changes we made → SET MARKS = MARKS + 2
condition → WHERE MARKS > 90

Eg ②:- update marks of student to 93 of roll no: 4

SOL:

```
UPDATE STUDENTS
SET MARKS = 93
WHERE roll no = 4 ;
```

③ GRADE 'B' for student of marks blw 80 to 90

SOL:

```
UPDATE STUDENTS
SET GRADE = "B"
WHERE MARKS BETWEEN 80 AND 90
```

④ DELETE: To delete existing rows.

```
DELETE FROM table_name
WHERE condition;
```

Page No:
Date:

Removing student with Less Than 35 marks.

Eg:
```
DELETE FROM Student
WHERE MARKS < 35;
```

To ~~remove~~ Delete entire data from table.

```
DELETE FROM table_name;
```

Revising Foreign Key:- We can see foreignkey usage properly in MySQL as well in form of EER Diagram

(1) table which has primary key in a relation of table, it is "parent table."

(2) Table which uses parent key of another table, in their table, it is "Child Key".

Eg:

Department

id	name
101	Science
102	Hindi
103	Social
104	English

Foreign Key

Teacher

foreignKey (FK)

id	name	dept_id
101	-	103
102	-	101
103	-	102
104	-	103

Parent Table

Child table.

Page No:
Date:

⑧ Cascading in Foreign Key:-

① On Delete Cascade:-

When we create a foreign key using this option, it deletes the referencing rows in the child table, when the reference row is deleted in the parent table which has primary key.

② On update cascade:-

The referencing rows are updated in the child table when the refere row is updated in the parent table which has primary key.

Student

```
CREATE TABLE ~~Stude~~ Child-table{
    col-1 ____
    col-2 ____
    FOREIGN KEY (COURSE ID) REFERENCES
                                Course(Id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
```

Simply; when we use foreign key, we use Delete (&) Update CASCADE.

Page No :
Date :

TABLE RELATED QUERIES:-

ALTER:-

1. ADD column:-

```
ALTER TABLE table-name
ADD COLUMN column-name datatype constraint;
```

Eg: ALTER TABLE Students
ADD COLUMN age INT NOT NULL;

2. DROP column:-

```
ALTER TABLE table-name
DROP column column-name;
```

Eg:-
```
ALTER TABLE Students
DROP column age;
```

3. RENAME Table:-

```
ALTER TABLE table-name
RENAME TO new-table-name;
```

Eg:
```
ALTER TABLE Students
RENAME TO Student-Info;
```

4. CHANGE COLUMN:- (rename)

```
ALTER TABLE table-name
CHANGE COLUMN old-name new-name new-datatype
new-constraint;
```

Page No :
Date :

Eg:- Id → Student Id.

```
ALTER TABLE students
CHANGE COLUMN Id StudentId INT PRIMARY KEY;
```

(5.) Modify Column:- (Modify datatype/constraint)

```
ALTER TABLE table_name
MODIFY col_name new_datatype new_constraint;
```

Eg:-

```
ALTER TABLE Student
MODIFY age VARCHAR(2);
```

TRUNCATE:- TO delete table's data

```
TRUNCATE TABLE table_name;
```

DROP	TRUNCATE
↓	↓
delete table	remove data from table still table exists.

Page No :
Date :

Joins in SQL:-

"Joins" is used to "combine rows" from two (or) more tables" based on a related column between them.

Eg:- 2 Tables

Employee (A)

id	name
101	Rahul
102	bhuvan

Salary (B)

id	Salary
102	25,000
103	21,000

(common column)

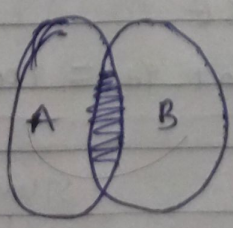
Here, id's column is common for employee (&) salary tables.

Now, we can combine rows of these tables (&) name and salary of id (102).

Types of Join (2):-

(3 types)

Inner Join

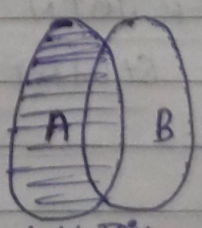


A & B are tables.

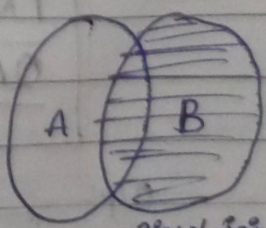
A → left side table

B → Right side table.

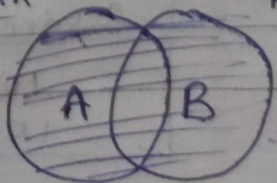
Outer Joins



Left Join



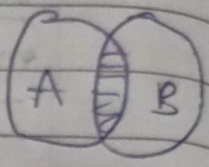
Right Join



Full Join.

Page No:
Date:

1. **Inner Join** :- Returns records that have matching values in both tables.



Syntax:-

```
SELECT Column(s)
FROM table A
INNER JOIN table B
ON tableA. col-name = table B. col_name
```

Eg:- Student

Student_id	name
101	adam
102	bob
103	casey

Course.

Student_id	Course
102	english
105	maths
103	science
107	Physics

SQL:

```
SELECT *
FROM Student
INNER JOIN COURSE
ON Student.student_id = course.student_id
```

O/P:-

Student_id	name	Student_id	Course
102	bob	102	english
103	casey	103	science

Page No:
Date:

To Simplify, we can use alias ⟺ as
(alternative name for Tables).

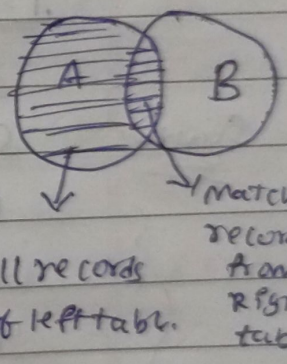
SQL:
```
SELECT *
FROM Student as S
INNER JOIN COURSE as C
ON S.Student_Id = C.Student_id;
```

(2.) Left Join:-

Returns all records from the left table, and matched records from the right table.

→ FROM table-A
↓ left table

→ left Join table-B
↓ Right table.



Order should be proper; If we interchange tables, values will change as well.

Syntax

```
SELECT column(s)
FROM tableA
LEFT JOIN tableB
ON tableA.col-name = tableB.col-name;
```

Page No:
Date:

eg:

Student (Left)

Student_id	name
101	adam
102	bob
103	casey

Course (Right)

Student_id	course
102	----
105	----
103	----
107	----

complete left table. (+) matched records.

Result:

Student_id	name	course
101	adam	null → not found in course table.
102	bob	english
103	casey	science

Syntax:

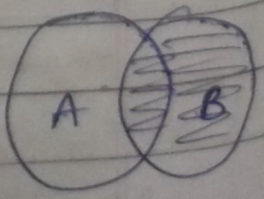
```
SELECT *
FROM student as S
LEFT JOIN course as C
ON S.student_id = C.student_id;
```

If we interchange, result will be Right Join

Student_id	course	name
102	english	bob
105	science	NULL
103	math	casey
107	physics	NULL

3. Right Join:

Same like left Join, but it return, complete right table with matched records from left table.



Page No :
Date :

Syntax:-
```
SELECT *
FROM student as S
RIGHT JOIN course as C
ON S.student_id = C.student_id;
```

O/P:-

If we interchange tables;
```
FROM course
RIGHT JOIN student
```
result will be left JOIN.

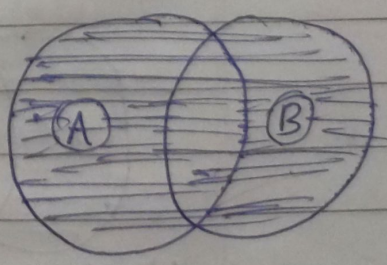
Note:- If, we "interchange" the tables in a LEFT JOIN, it becomes logically the same as a RIGHT JOIN, vice versa.

(A) LEFT JOIN (B) = (B) RIGHT JOIN (A)

4. FULL JOIN:- Returns all records when there is a match in either left (or) right table.

Simply;

LEFT JOIN
UNION
RIGHT JOIN



Union - It give unique values.
So, it give the common column only once.

Page No:
Date:

Syntax:

```
SELECT *
FROM Student as a
LEFT JOIN course as b
ON a.id = b.id
```

LEFT JOIN

FULL JOIN

```
UNION
SELECT *
FROM Student as a
LEFT JOIN course as b
ON a.id = b.id
```

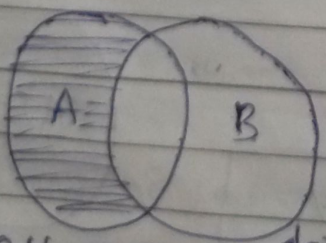
UNION

RIGHT JOIN

O/P:

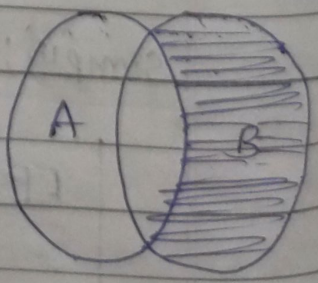
id	name	id	course
103	Casey	103	Science
102	boy	102	English
101	adam	NULL	NULL
NULL	NULL	105	Math
NULL	NULL	107	computer science.

④ Left Exclusive Join



NON-Related data present in A, Compared to B.

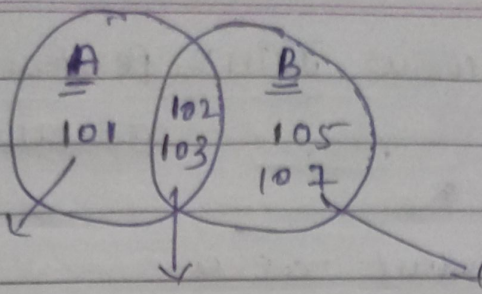
RIGHT Exclusive Join



NON-Related data of B

Page No:
Date:

Eg:-



(1.) Left Exclusive Join:- We find Left Join, then we remove Id's which are NULL

for table A; Id's → 105, 107 are NULL, ~~so we get~~

table B; Id's → 101 is NULL.

Sol:-

```
SELECT *
FROM Student as a
LEFT JOIN course as b
ON a.id = b.id
WHERE b.id IS NULL;
```

check Values which are NULL.

Value 101; b.id is NULL. So, we get Left Exclusive Join.

Olp:-

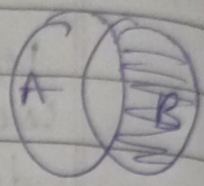
Id	name	Id	course
101	adam	NULL	NULL

Page No:
Date:

④ Right Exclusive JOIN:- perform Right JOIN and id IS NULL

Right Join

```
SELECT *
FROM Student as a.
RIGHT JOIN Course as b
ON a.id = b.id
WHERE a.id IS NULL
```

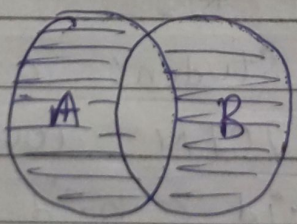


ids 105, 107 are NULL

o/p:-

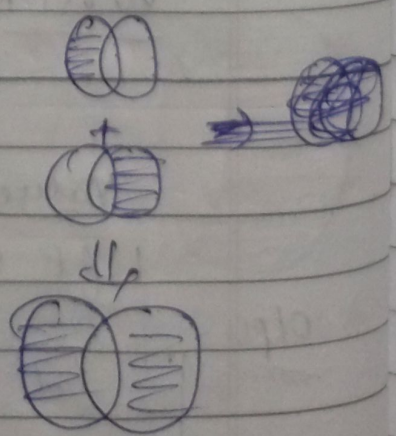
id	name	id	Course
NULL	NULL	105	maths
NULL	NULL	107	Physics.

⑤ Full Exclusive JOIN:-



Combination of left and Right exclusives.

Left Exclusive JOIN
UNION
RIGHT Exclusive JOIN.



Page No:
Date:

(4) Self Join:- It is a regular join but the table is joined with itself.

Syntax:

```
SELECT Column(s)
FROM table as a
JOIN table as b
ON a.col-name = b.col-name;
```

} same table

Eg:- Employee

id	name	manager_id
101	adam	103 (casey)
102	bob	104 (donald)
103	casey	null
104	donald	103 (casey)

↑ same table we use it as a & b

Casey
↓ manager of
adam
donald
↓ manager of
bob

→ Now, we need to select a.name as manager_name to take manager's list (&) b.name to take employee names.

Employee

Page No:
Date:

Employee names who are managers

```
SELECT a.name as manager_name, b.name
FROM employee as a
JOIN employee as b
ON a.id = b.manager_id;
```

both are give table

employee id = manager id

O/P:-

manager_name	name
Casey	addm
donald	bob
casey	donald

(3) (4) (5) UNION:- It is used to combine the result-set of two (or) more SELECT statments.

To use it: (1) every SELECT should have same no. of columns.

(2) Columns must have similar datatype.

(3) Columns in every SELECT should be in same order.

Syntax:

```
SELECT (column(s)) FROM tableA
UNION
SELECT (column(s)) FROM tableB
```

Page No:
Date:

- Eg:
```
SELECT name FROM employee;
UNION
SELECT name FROM employee;
```

O/P:-

name
adam
bob
casey
donald

Here, UNION Removes all duplicate values (i.e) provide unique values.

⑤ UNION ALL:- It also provides duplicate values as well.

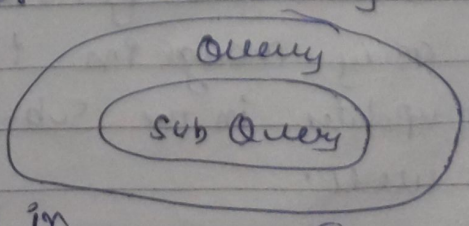
```
UNION ALL
```

O/P:-

name
adam
bob
casey
donald
adam
bob
casey
donad

⑥ SQL Sub-Queries:-

A SubQuery (or) Inner Query (or) a Nested query is a query within another SQL Query.



most preferable ↑

We perform subqueries in ③ types ① SELECT ② FROM ③ WHERE

Page No:
Date:

Syntax:
```
SELECT  Column(s)
FROM    table_name
WHERE   col_name operator
(Sub Query);
```
(+, =, >, <, >=, <=, !=)

Example:

Student

roll no	name	marks
101	anil	78
102	bhumika	93
103	Chetan	85
104	dhruv	96
105	emanuel	92
106	farah	82

(1) Get names of all students who scored more than class average.

Syntax:

Query:
```
SELECT (name, marks)
FROM Student
WHERE marks > operator
SELECT AVG(marks) FROM Student;
```
(87.366)

Sub-Query

NOTE: Sub-Queries are dynamic. If there is any change in table, automatically it updates in the Sub-Query result as well.

O/P:

name	marks
bhumika	93
dhruv	96
emanuel	92

Page No :
Date :

(2.) Find the names of all students with even roll numbers.

(A.) Query:

```
SELECT name, rollno
FROM student
WHERE rollno IN (
    SELECT rollno
    FROM student
    WHERE rollno % 2 = 0);
```

IN → operator.

Sub-query

• Even roll numbers.

Eg (3):- Find the highest marks from Mumbai city.

Sub-query through FROM :-

```
SELECT max(marks)
FROM (SELECT * FROM student WHERE
      city = "Mumbai") AS temp;
```

Alias

Sub-query.

general way:-

WHERE:

```
SELECT max(marks)
FROM student
WHERE city = "Mumbai";
```

Page No :
Date :

MY SQL Views :-

A View is a Virtual table based on the result-set of an SQL statement.

Student :-

roll no	name	marks	grade	city

Now, I want to create a View which having only rollno, name, marks only.

View name

CREATE VIEW View1 AS
SELECT rollno, name, marks FROM Stdet;

SELECT * FROM View1

O/P:

rollno	name	marks

DROP View :

DROP VIEW View1;

Page No:
Date:

Extra

DCL (Data control Language):-

Data control language in SQL is used to control access (&) permissions to data within the database.

① GRANT:-

GRANT SELECT, INSERT ON Employees TO John;

TO → GRANT

commands for accessing on Employee table.

to the user "John".

GRANT ALL:- To allow all command access to an user.

GRANT ALL PRIVILEGES ON Employees TO admin;

(same)

GRANT ALL ON Employees TO admin;

Here; ALL (&) ALL PRIVILEGES Perform similar task; to grant all command access to user (admin).

② REVOKE:- To remove previously granted access privileges from user.

REVOKE INSERT ON Employee FROM John;

- INSERT → commands to be revoke.
- Employee → table
- John → user name.

Page No:
Date:

To take all ;

ALL (or)

REVOKE ALL PRIVILEGES ON Customers
FROM admin;

(3) DENY:

↳ It is used to explicitely block specific permission for a user on database object.

DENY SELECT ON Employees TO John;

Both "DENY" (&) "REVOKE"

↓ Block the access.

↘ remove the previously grant permission.

TCL (Transaction Control Language):-

Transaction: sequence of one (or) more data base operations that are executed as a single logical unit

(Insert, update, delete)

Page No :
Date :

(1) To begin "Transaction"

```
BEGIN TRASACTION;
INSERT INTO employees VALUES (102, "Ram", "HR");
UPDATE employee SET department = 'finance" WHERE emp id = 201;
```

(2) SAVEPoint :- we can roll back to this point later.

```
SAVEPOINT before-update;
UPDATE employees - ;
```

(3) ROLLBACK TO :- undoes the last update but keeps previous insert intact.

```
ROLLBACK TO before- update;
```

(4) COMMit :- All remaining changes will Saved permanently to database.

```
COMMIT;
```